Masterbikes: Documentación Swagger, JUnit+Mockito & HATEOAS

Integrantes:

- Hector Águila
- Fabián Lecaros
- Franco Ruz
- Cesar Veliz

Introducción

A. Documentación del Sistema

A.1. Diagrama de Arquitectura

A.2. Documentación Swagger (OpenApi)

A.2.1. catalogo-service

A.2.2. sucursal-service

A.2.3. inventario-service

A.2.4. venta-service

B. Plan de Pruebas - JUnit + Mockito

B.1.1. AccesorioServiceTest

B.1.2. BicicletaServiceTest

B.2. sucursal-service

B.2.1. EmpleadoServiceTest

B.2.2. SucursalServiceTest

B.3. inventario-service

B.3.1. InventarioServiceTest

B.3.2. MovimientoInventarioServiceTest

B.4. venta-service

B.4.1. FacturaServiceTest

B.4.2. VentaServiceTest

C. Implementación de HATEOAS

C.1. catalogo-service.AccesorioController

C.1.1. Clases involucradas

C.1.2. Modificaciones principales

D. Conclusiones Finales

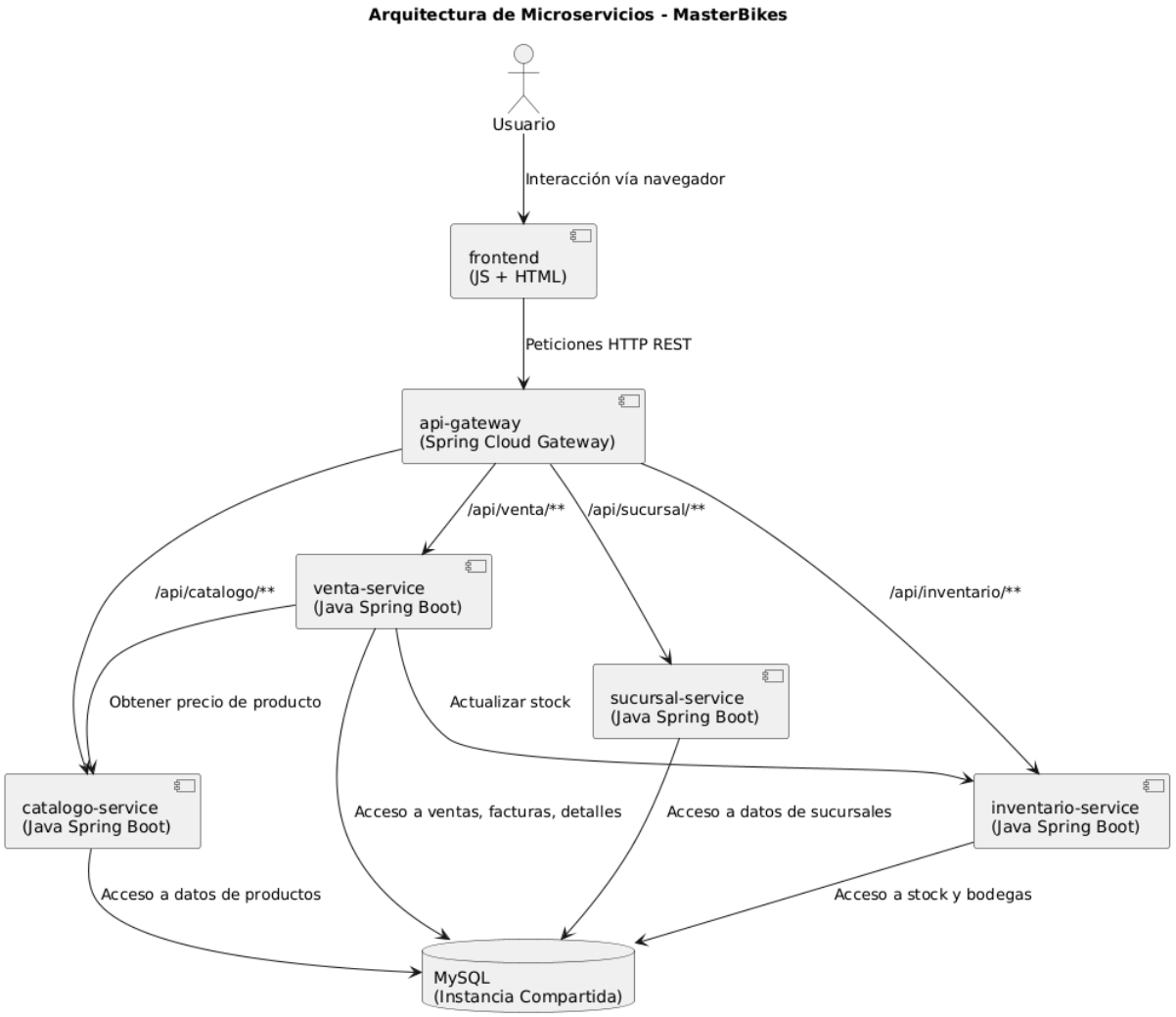
Introducción

El siguiente documento describe la implementación en el proyecto de Masterbikes de la documentación generada mediante Swagger, pruebas unitarias usando JUnit-Mockito y de HATEOAS en uno de sus controladores.

Se puede acceder a la última versión del repositorio del proyecto en el siguiente enlace: https://github.com/flecaros84/Github/tree/main/masterbikes

A. Documentación del Sistema

A.1. Diagrama de Arquitectura



La arquitectura de microservicios del proyecto se compone de 4 microservicios y 3 componentes adicionales:

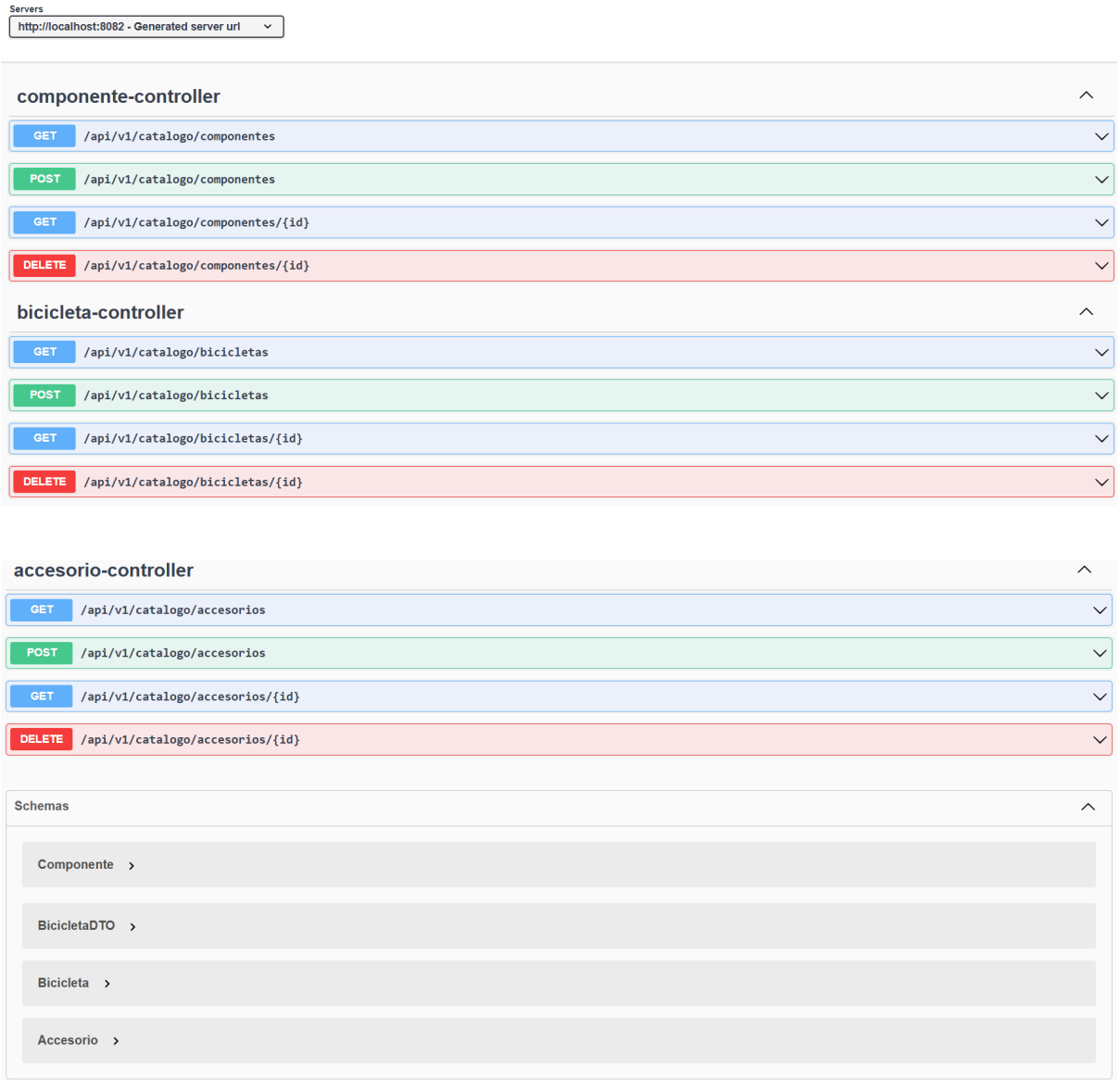
- catalogo-service: Microservicio que gestiona los productos, sus características y precios.
- sucursal-service: Microservicio que gestiona la información de sucursales y empleados.
- inventario-service: Microservicio que gestiona el inventario de productos.
- venta-service: Microservicio que gestiona la venta.
- api-gateway: Centraliza las llamadas desde el frontend

- Base de datos MySql: Encargada de la persistencia de datos, de momento gestionada localmente a través de Laragon.
- frontend: Interfaz con el usuario

A.2. Documentación Swagger (OpenApi)

A.2.1. catalogo-service

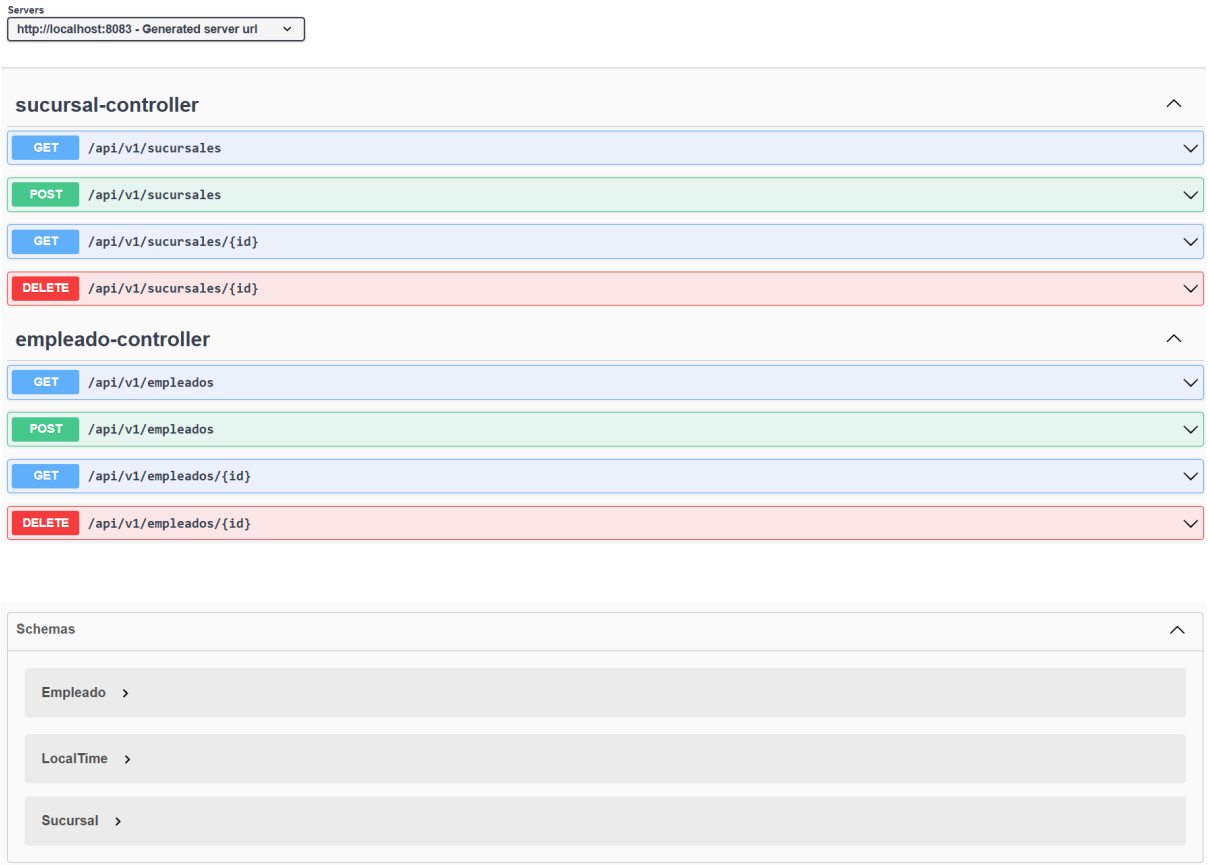
https://github.com/flecaros84/Github/tree/main/masterbikes/catalogo-service/src/main/java/masterbikes/catalogo_service/controller



Los endpoint del microservicio catalogo-service implementan las funcionalidades requeridas para manejar la información de los distintos productos.

A.2.2. sucursal-service

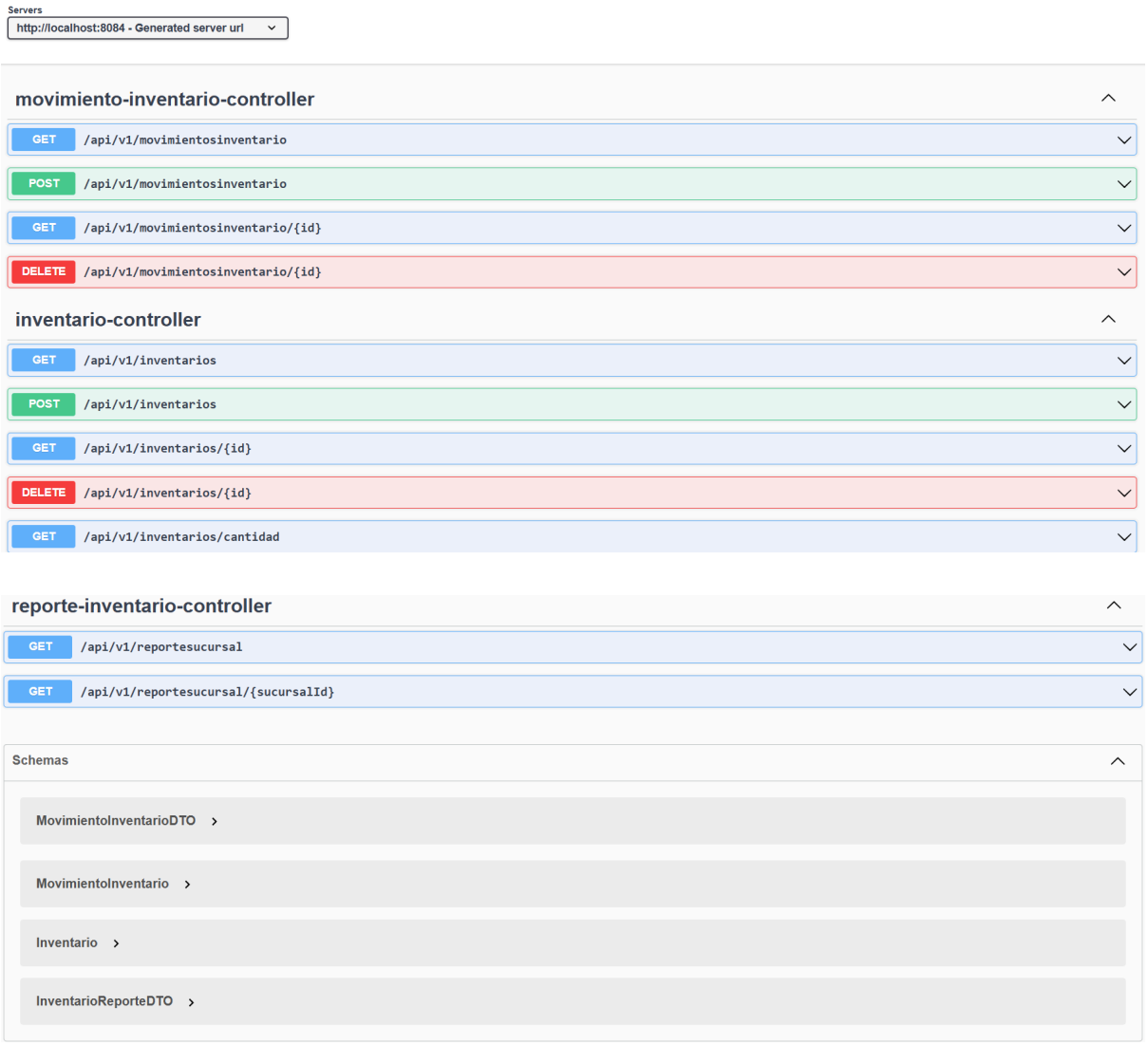
https://github.com/flecaros84/Github/tree/main/masterbikes/sucursal-service/src/main/java/masterbikes/sucursal_service/controller



Los endpoint del microservicio sucursal-service implementan las funcionalidades requeridas para manejar la información de las sucursales, incluyendo esto la información de los empleados que trabajan en ellas, con fin permitir extender en el futuro su integración a un servicio de autenticación y/o permisos (aún no implementado).

A.2.3. inventario-service

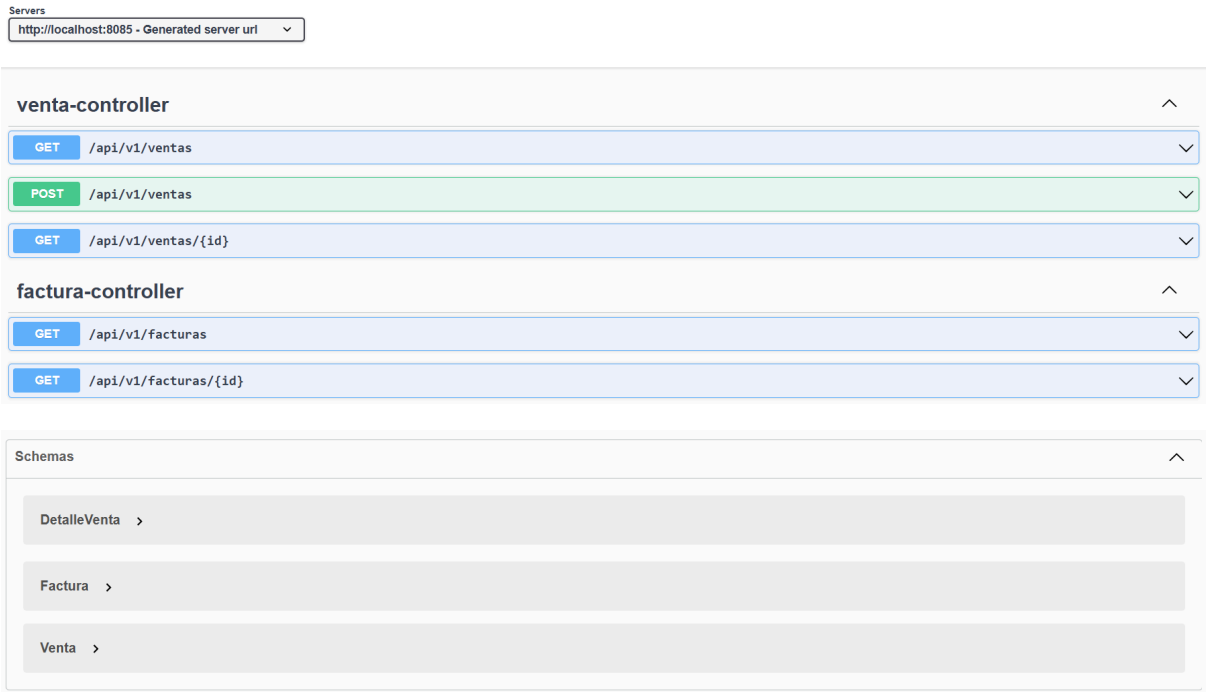
https://github.com/flecaros84/Github/tree/main/masterbikes/inventario-service/src/main/java/masterbikes/inventario_service/controller



Los endpoint del microservicio inventario-service implementan las funcionalidades requeridas para manejar los inventarios, incluyendo esto tanto el inventario en sí (stock) como el registro de movimiento de inventario. Se incluyó además un reporte que se comunica con los microservicios de catalogo y sucursal permitiendo obtener un mayor detalle del stock (por ejemplo obtener el nombre del producto desde catalogo-service y el nombre de la sucursal desde sucursal-service).

A.2.4. venta-service

https://github.com/flecaros84/Github/tree/main/masterbikes/venta-service/src/main/java/masterbikes/venta_service/controller



Los endpoint del microservicio venta-service implementan las funcionalidades requeridas para la gestión de ventas y facturas. La venta incluye un detalle de la misma, compuesto de varios productos y unidades.

B. Plan de Pruebas - JUnit + Mockito

B.1. catalogo-service

https://github.com/flecaros84/Github/tree/main/masterbikes/catalogo-service/src/test/java/masterbikes/catalogo_service/service

B.1.1. AccesorioServiceTest

Clases involucradas

- AccesorioService: clase bajo prueba.
- AccesorioRepository: simulado con Mockito como dependencia.
- Accesorio: modelo de dominio utilizado como dato de prueba.

Descripción de las pruebas implementadas

1. **listarTodos_deberiaRetornarListaDeAccesorios**
   Verifica que el método findAll() del servicio devuelva correctamente una lista de accesorios cuando el repositorio retorna datos simulados.

2. **guardarAccesorio_deberiaRetornarAccesorioGuardado**
   Comprueba que el método save() del servicio retorne el accesorio guardado, con los valores esperados, y que se invoque correctamente el repositorio.

3. **buscarPorId_existente_deberiaRetornarAccesorio**
   Valida que se recupere un accesorio cuando se consulta por un ID existente.

4. **buscarPorId_inexistente_deberiaRetornarNull**
   Confirma que al consultar un ID no existente, el servicio retorne null, simulando una ausencia de datos en el repositorio.

5. **eliminarPorId_deberiaLlamarAlRepositorio**
   Verifica que el método delete() del servicio invoque correctamente al método deleteById() del repositorio con el ID especificado.

B.1.2. BicicletaServiceTest

Clases involucradas

- BicicletaService: clase bajo prueba, contiene la lógica de validación y guardado de bicicletas.
- BicicletaRepository y ComponenteRepository: dependencias simuladas con Mockito.
- Bicicleta y Componente: modelos utilizados para construir escenarios de prueba.

Descripción de las pruebas implementadas

1. **listarTodas_deberiaRetornarListaDeBicicletas**
   Verifica que el método findAll() devuelva correctamente una lista de bicicletas, utilizando datos simulados del repositorio.

2. **guardarBicicletaValida_deberiaPasarValidacionesYGuardar**
   Comprueba que una bicicleta con componentes compatibles (marco, rueda y freno) sea validada correctamente y guardada sin errores.

3. **guardarBicicletaConRuedaIncompatible_deberiaLanzarExcepcion**
   Simula el caso en que la rueda no es compatible con el marco (por diámetro o tipo de uso), y verifica que se lance una excepción con el mensaje correspondiente.

4. **guardarBicicletaConFrenoIncompatible_deberiaLanzarExcepcion**
   Valida que se rechace una bicicleta cuando el tipo de freno no es compatible con el tipo de freno soportado por el marco.

B.1.3. ComponenteServiceTest

Clases involucradas

- ComponenteService: clase bajo prueba que encapsula la lógica relacionada con el manejo de componentes.
- ComponenteRepository: dependencia simulada con Mockito.
- Componente: modelo utilizado para los escenarios de prueba.

Descripción de las pruebas implementadas

1. **listarTodos_deberiaRetornarListaDeComponentes**
   Verifica que el método findAll() devuelva correctamente una lista de componentes simulados, y que el repositorio haya sido invocado.

2. **guardarComponenteValido_deberiaRetornarComponenteGuardado**
   Comprueba que el método save() almacene correctamente un componente válido, y que los valores de retorno coincidan con los esperados.

3. **buscarPorId_existente_deberiaRetornarComponente**
   Valida que el servicio retorne un componente cuando el ID consultado existe en el repositorio.

4. **buscarPorId_inexistente_deberiaRetornarNull**
   Simula un caso donde no existe un componente con el ID consultado, y comprueba que el servicio retorne null.

5. **eliminarPorId_deberiaLlamarAlRepositorio**
   Verifica que al llamar al método delete() del servicio, se invoque correctamente el método deleteById() del repositorio.

B.2. sucursal-service

https://github.com/flecaros84/Github/tree/main/masterbikes/sucursal-service/src/test/java/masterbikes/sucursal_service/service

B.2.1. EmpleadoServiceTest

Clases involucradas

- EmpleadoService: clase bajo prueba, que gestiona la lógica relacionada con empleados en una sucursal.
- EmpleadoRepository: dependencia simulada mediante Mockito.
- Empleado y Sucursal: modelos utilizados como datos de prueba.

Descripción de las pruebas implementadas

1. **listarEmpleados_deberiaRetornarLista**
   Verifica que el método findAll() devuelva correctamente una lista de empleados, utilizando una respuesta simulada del repositorio.
2. **guardarEmpleadoValido_deberiaRetornarEmpleadoGuardado**
   Comprueba que el método save() almacene correctamente un empleado con datos válidos (incluyendo su sucursal asociada), y que los valores retornados coincidan con lo esperado.
3. **buscarEmpleadoExistente_deberiaRetornarEmpleado**
   Valida que se retorne un empleado cuando se consulta por un ID existente en el repositorio.
4. **buscarEmpleadoInexistente_deberiaRetornarNull**
   Simula la búsqueda de un ID inexistente y verifica que el método findById() retorne null.
5. **eliminarEmpleadoExistente_deberiaEjecutarDelete**
   Verifica que al llamar al método delete() del servicio, se invoque correctamente el método deleteById() del repositorio con el ID correspondiente

B.2.2. SucursalServiceTest

Clases involucradas

- SucursalService: clase bajo prueba que maneja la lógica relacionada con las sucursales.
- SucursalRepository: dependencia simulada mediante Mockito.
- Sucursal: modelo utilizado como dato de prueba para los distintos escenarios.

Descripción de las pruebas implementadas

1. **listarSucursales_deberiaRetornarLista**
   Verifica que el método findAll() retorne correctamente una lista de sucursales utilizando datos simulados del repositorio.

2. **guardarSucursalValida_deberiaRetornarSucursalGuardada**
   Comprueba que el método save() almacene correctamente una sucursal con todos los campos relevantes definidos (dirección, comuna, horarios, etc.), y que los valores retornados sean los esperados.

3. **buscarSucursalExistente_deberiaRetornarSucursal**
   Valida que el servicio retorne una sucursal específica cuando se consulta por un ID existente.

4. **buscarSucursalInexistente_deberiaRetornarNull**
   Simula la búsqueda de una sucursal inexistente por ID, y verifica que el resultado sea null.

5. **eliminarSucursalExistente_deberiaEjecutarDelete**
   Verifica que al ejecutar el método delete(), se invoque correctamente el método deleteById() del repositorio con el ID indicado.

B.3. inventario-service

https://github.com/flecaros84/Github/tree/main/masterbikes/inventario-service/src/test/java/masterbikes/inventario_service/service

B.3.1. InventarioServiceTest

Clases involucradas

- InventarioService: clase bajo prueba que gestiona la lógica de inventario por producto y sucursal.
- InventarioRepository: dependencia simulada con Mockito.
- Inventario: modelo utilizado como dato de prueba en los distintos escenarios.

Descripción de las pruebas implementadas

1. **listarTodo_deberiaRetornarListaDeInventario**
   Verifica que el método findAll() retorne correctamente una lista de registros de inventario simulados.
2. **guardarInventario_deberiaRetornarInventarioGuardado**
   Comprueba que el método save() almacene correctamente un nuevo registro de inventario, y que los valores retornados coincidan con lo esperado.
3. **buscarPorId_existente_deberiaRetornarInventario**
   Valida que se retorne un registro de inventario cuando se consulta por un ID existente.
4. **buscarPorId_inexistente_deberiaRetornarNull**
   Simula la búsqueda de un ID inexistente y verifica que el método retorne null.
5. **eliminarPorId_deberiaLlamarARepositorio**
   Verifica que al ejecutar el método deleteById(), se invoque correctamente el repositorio con el ID especificado.

6. **buscarPorProductoYTipo_deberiaRetornarLista**
   Valida que el método findByProductoIdAndTipoProducto() funcione correctamente y devuelva los registros de inventario correspondientes al ID de producto y tipo indicados.

B.3.2. MovimientoInventarioServiceTest

Clases involucradas

- MovimientoInventarioService: clase bajo prueba, encargada de aplicar movimientos de entrada, salida o ajuste sobre el inventario.
- MovimientoInventarioRepository y InventarioRepository: dependencias simuladas mediante Mockito.
- MovimientoInventarioDTO: objeto de transferencia de datos utilizado para registrar movimientos.
- Inventario y MovimientoInventario: modelos utilizados como datos de prueba.

Descripción de las pruebas implementadas

1. **registrarEntrada_deberiaIncrementarInventarioYGuardarMovimiento**
   Valida que al registrar un movimiento de tipo **ENTRADA**, la cantidad en inventario se incremente correctamente y se guarde el movimiento asociado.

2. **registrarSalida_conStockSuficiente_deberiaReducirInventarioYGuardarMovimiento**
   Verifica que una **SALIDA** se procese correctamente cuando hay suficiente stock disponible, y que se refleje la disminución en la cantidad del inventario.

3. **registrarSalida_conStockInsuficiente_deberiaLanzarExcepcion**
   Simula una salida mayor al stock disponible y verifica que se lance una excepción con el mensaje "Stock insuficiente".

4. **registrarAjuste_deberiaActualizarCantidadDeInventario**
   Comprueba que un movimiento de tipo **AJUSTE** modifique directamente la

cantidad del inventario según el valor entregado, incluso si es negativo.

5. **registrarMovimiento_conTipoInvalido_deberiaLanzarExcepcion**
   Verifica que se lance una excepción cuando el tipo de movimiento no corresponde a ninguno de los casos válidos ("ENTRADA", "SALIDA", "AJUSTE").

B.4. venta-service

https://github.com/flecaros84/Github/tree/main/masterbikes/venta-service/src/test/java/masterbikes/venta_service/service

B.4.1. FacturaServiceTest

Clases involucradas

- FacturaService: clase bajo prueba que gestiona las operaciones de facturación.
- FacturaRepository: dependencia simulada mediante Mockito.
- Factura y Venta: modelos utilizados como datos de prueba para los distintos escenarios.

Descripción de las pruebas implementadas

1. **listarFacturas_deberiaRetornarLista**
   Verifica que el método listar() retorne correctamente una lista de facturas, utilizando una respuesta simulada del repositorio.

2. **guardarFacturaValida_deberiaRetornarFacturaGuardada**
   Comprueba que el método guardar() almacene correctamente una factura válida con todos los campos necesarios, y que el resultado devuelto coincida con lo esperado.

3. **buscarFacturaExistente_deberiaRetornarFactura**
   Valida que el método buscar() retorne una factura cuando se consulta por un ID existente.

4. **buscarFacturaInexistente_deberiaRetornarNull**
   Simula la búsqueda de una factura con un ID inexistente, y verifica que el método retorne null.

5. **eliminarFacturaExistente_deberiaEjecutarDelete**
   Verifica que el método eliminar() invoque correctamente la eliminación de una factura a través del repositorio, usando el ID correspondiente.

B.4.2. VentaServiceTest

Clases involucradas

- VentaService: clase bajo prueba, que implementa la lógica principal para generar ventas, consultar precios y verificar inventario.
- VentaRepository: dependencia simulada con Mockito, utilizada para almacenar la venta generada.
- RestTemplate: objeto simulado para realizar llamadas a los microservicios de catálogo e inventario.
- Venta, DetalleVenta: modelos de dominio utilizados como entradas de prueba.
- ProductoBaseDTO, InventarioDTO: objetos utilizados como respuesta simulada desde servicios externos.

Descripción de las pruebas implementadas

1. **generarVentaValida_deberiaCalcularTotalYGuardarVenta**
   Verifica que el método generarVenta() calcule correctamente el total de la venta y sus valores tributarios, cree una factura interna y almacene la venta, tras consultar el precio unitario y confirmar disponibilidad en el inventario.

2. **generarVenta_conProductoNoExistente_enCatalogo_deberiaLanzarExcepcion**
   Simula un escenario en el que el producto especificado no se encuentra en el microservicio de catálogo. Se espera que el servicio lance una excepción con el mensaje "Producto no encontrado en catálogo".

3. **generarVenta_conInventarioInexistente_deberiaLanzarExcepcion**
   Valida que si no existe stock registrado para el producto en la sucursal indicada, el servicio lance una excepción indicando que no hay inventario disponible.

C. Implementación de HATEOAS

Se hizo esta prueba en otro repositorio dado que es incompatible con Swagger, y para evitar riesgos de inutilizar parte del código.

C.1. catalogo-service.AccesorioController

Para enriquecer las respuestas REST del microservicio con enlaces navegables (siguiendo el principio HATEOAS), se implementó una estructura que separa la lógica de negocio de la representación de los recursos. Esto permite que el cliente no solo reciba los datos del accesorio, sino también los enlaces útiles relacionados (por ejemplo, para ver el accesorio, ver todos, etc.). Se hizo esta implementación en la clase AccesorioController, por simplicidad dado que el modelo de Accesorio es uno de los más sencillos del proyecto.

C.1.1. Clases involucradas

https://github.com/flecaros84/Github/tree/main/masterbikes%20HATEOAS/catalogo-service/src/main/java/masterbikes/catalogo_service/hateoas

- **AccesorioModel** (hateoas):
  Es una clase que representa la versión HATEOAS del objeto Accesorio. Contiene los mismos datos que la entidad original, más una colección de enlaces (links). Extiende RepresentationModel.

- **AccesorioAssembler** (hateoas):
  Se encarga de convertir instancias de Accesorio en objetos AccesorioModel, agregando enlaces relevantes a cada recurso. Implementa la interfaz RepresentationModelAssembler.

https://github.com/flecaros84/Github/blob/main/masterbikes%20HATEOAS/catalogo-service/src/main/java/masterbikes/catalogo_service/controller/AccesorioController.java

- **AccesorioController** (controller):
  Fue modificado para que sus métodos devuelvan objetos de tipo AccesorioModel o CollectionModel<AccesorioModel>, en lugar de entidades Accesorio puras. Esto permite enviar respuestas HATEOAS al cliente.

C.1.2. Modificaciones principales

- Se inyectó el AccesorioAssembler en el controller.

- Los métodos GET (findAll, findById) y POST (save) fueron actualizados para retornar objetos AccesorioModel en lugar de entidades.

- Se agregó un enlace self a cada accesorio y un enlace general para volver al listado de todos los accesorios.

D. Conclusiones Finales

La utilización de las herramientas revisadas en este informe son de gran utilidad en entornos profesionales de desarrollo en Java Spring.

En primer lugar, con Swagger el desarrollador puede obtener una documentación básica y robusta de forma relativamente sencilla, permite testear endpoints, y además ofrece opciones de personalización.

En cuanto a JUnit-Mockito, es una herramienta fundamental para testear servicios, lo que permite dar certezas del funcionamiento de estos dentro de la lógica de la aplicación sin tener que hacerlo directamente en los endpoints.

Finalmente HATEOAS permite enriquecer las respuestas del REST con enlaces navegables, facilitando el acceso a los endpoints cuando sea necesario. De las tres herramientas fue la que menos utilidad le vimos intuitivamente, pero esto puede ser sin duda consecuencia de nuestra relativa inexperiencia en el desarrollo y no de que la herramienta no posea un valor en sí.

En resumen, lo aprendido complementa otros aprendizajes del semestre en Java Spring.